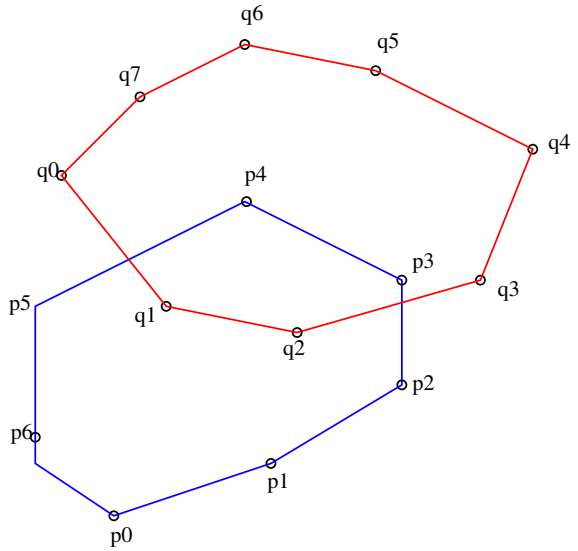
Geometriai algoritmusok 2

Szerző: Horváth Gyula
horvath@inf.elte.hu

2. Feladat: Két zárt konvex poligonok metszete

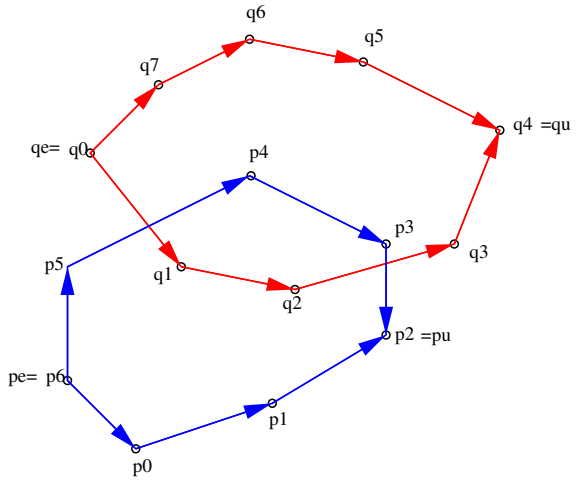
Bemenet: $P = (p_0, \ldots, p_{n-1})$ és $Q = (q_0, \ldots, q_{m-1})$ zárt konvex poligonok a csúcsaik órajárással ellentétes felsorolásával.
Döntsük el, hogy a két poligon metszi-e egymást! Ha van metszés, akkor adjunk meg egy metsző oldalpárt!

Megoldás



1. ábra

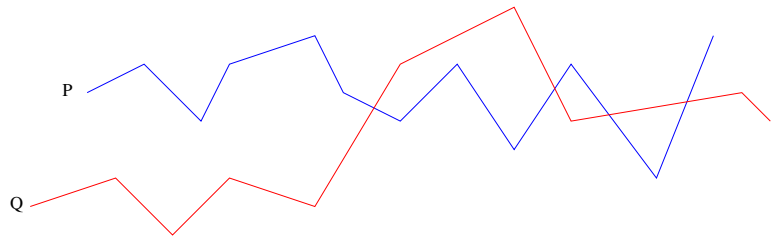
Irányítsuk a poligonok oldalait úgy, hogy x-koordinátában növekvőek legyenek. Legyen pe az első, azaz a legkisebb x koordinátájú pont, pu pedig az utolsó, azaz a legnagyobb x koordinátájú pont. Hasonlóan határozzuk meg a qe és qu pontokat.
látható, hogy a P (zárt) poligon felbontható egy Palso és egy Pfelo, a Q poligon pedig felbontható egy Qalso és Qfelso poligonra.
Nyilvánvaló, hogy ha van $u \in P$ és $v \in Q$ metsző oldalpár, akkor $u \in$ Palso vagy $u \in$ Pfelso, illetve $v \in$ Qalso vagy $v \in$ Qfels teljesül.



2. ábra

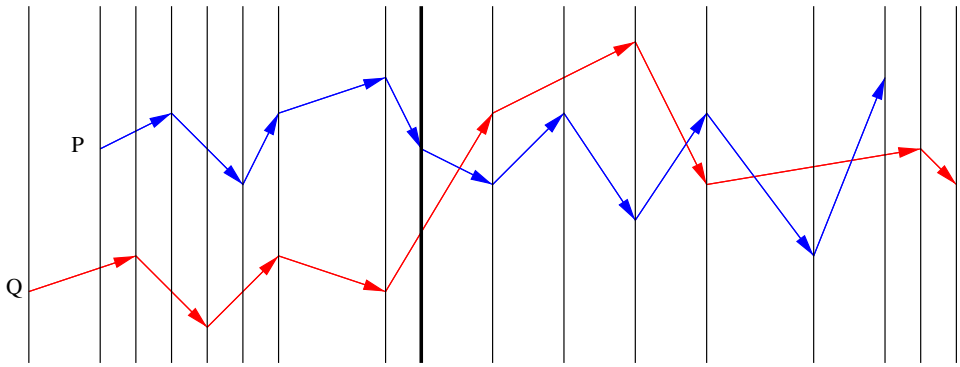
Az mondjuk, hogy egy $P = (p_1, p_2, \ldots, p_n)$ poligon **x-monoton**, ha $p_i.x < p_{i+1}.x$ minden $1 \leq i < n$-re.

Megjegyezzük, hogy az x-monoton poligon nem feltétlenül konvex.
Mind a négy poligon x-monoton, tehát a kiindulási probléma megoldása visszavezethető két x-monoton poligon metszetének meghatározására.
Megmutatjuk, hogy ha $P = (p_1, p_2, \ldots, p_n)$ és $Q = (q_1, q_2, \ldots, q_m)$ x-monoton poligonok, akkor $O(n + m)$ futási időben meghatározható a metszetük.



3. ábra

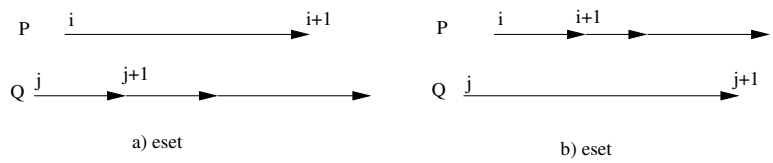
Alkalmazzuk a **pásztázó**, vagy más néven **seprő** egyenes módszerét: tekintsük az összes olyan y-tengellyel párhuzamos egyenest, amelyen rajta van a két poligonnak legalább egy pontja. Ezeket az x-koordináta értékeket a pásztázó egyenes **megállási helyeinek** nevezzük.
Minden e pásztázó egyeneshez, megállási helyhez tartozik egy **állapothalmaz**, azon poligon oldalak halmaza, amelyek metszik az e egyenest.
Esetünkben, mivel a monotonitásból következően teljesül, hogy minden állapothalmaz egy, vagy két elemet tartalmaz.



4. ábra

2.1. Állítás. *A* P *és* Q *x-monoton poligonoknak akkor és csak akkor metszi egymást, ha van olyan* e *pásztázó egyenes, hogy van olyan* op *és* $oq \in Q$ *szakasz, hogy* $op, oq \in Allapot(e)$ *és az* op *és az* oq *szakaszok metszők.*

A monotonitás miatt lineáris időben előállítható a megállási helyek rendezett sorozata, továbbá megadható minden megállási helyre az a két oldal, amelyek metszik az adott pásztázó egyenest.
Első lépésként határozzuk meg azt a legkisebb i és legkisebb j indexet, amelyekre teljesül, hogy a $[P[i].x, P[i+1].x]$ és a $[Q[j].x, Q[i+1].x]$ intervallumoknak van közös pontjuk. Tehát az első olyan megállási hely, amely állapotban van két poligon egy-egy oldala. Ezt követően minden lépésben i vagy j lép az ábrán megadott feltétel szerint, tehát ha $P[i+1] < Q[j+1]$ akkor i lép, egyébként j lép.
Az is látható, hogy ezzel a módszerrel az összes metsző oldalpárt meg tudjuk határozni.



5. ábra. a) esetben `j` lép, b) esetben `i` lép.

```
bool MonotonMetsz(vector<Pont>&P, vector<Pont>& Q, int& a, int& b){
  int i=0, j=0;
  int n=P.size(), m=Q.size();
  while(i+1<n && P[i+1].x<Q[0].x) i++;
  while(j+1<m && Q[j+1].x<P[0].x) j++;
  while(i+1<n && j+1<m){
    if(SzakaszParMetsz(P[i],P[i+1], Q[j],Q[j+1])){
      a=P[i].azon; b=Q[j].azon;
      return true;
    }
    if(P[i+1].x<Q[j+1].x)
      i++;
    else
      j++;
  }
  return false;
}
```

```
bool KonvexMetsz(vector<Pont>& P, vector<Pont>& Q, int& a, int& b)
  int n=P.size(), m=Q.size();
  int pe=0,pu=0,qe=0,qu=0;
  for(int i=0;i<n;i++){
    if(P[i].x<P[pe].x || P[i].x==P[pe].x && P[i].y<P[pe].y )pe=i;
    if(P[i].x>P[pu].x || P[i].x==P[pu].x && P[i].y<P[pu].y )pu=i;
  }
  for(int i=0;i<n;i++){
    if(Q[i].x<Q[qe].x || Q[i].x==Q[qe].x && Q[i].y<Q[qe].y )qe=i;
    if(Q[i].x>Q[qu].x || Q[i].x==Q[qu].x && Q[i].y<Q[qu].y )qu=i;
  }
  //
  vector<Pont> Palso,Pfelso;
  vector<Pont> Qalso,Qfelso;
  int i=pe;
  while(i!=pu){Palso.pb(P[i]); i=(i+1)%n;}
  Palso.pb(P[pu]);
  i=pe-1; if(i<0) i=n-1;
  while(i!=pu){Pfelso.pb(P[i]); i--; if(i<0) i=n-1;}

  i=qe;
  while(i!=qu){Qalso.pb(Q[i]); i=(i+1)%n;}
  Qalso.pb(Q[qu]);
  i=(qu+1)%m;
  do{
    Qfelso.pb(Q[i]); i=(i+1)%m;
  }while(i!=qe);
  reverse(Qfelso.begin(), Qfelso.end());

  if(MonotonMetszt(Palso, Qalso, a, b))
    return true;
  if(MonotonMetszt(Palso, Qfelso, a, b))
    return true;
  if(MonotonMetszt(Pfelso, Qalso, a, b))
    return true;
  if(MonotonMetszt(Pfelso, Qalso, a, b))
    return true;
  return false;
}
```

2.1. Két poligon metszete

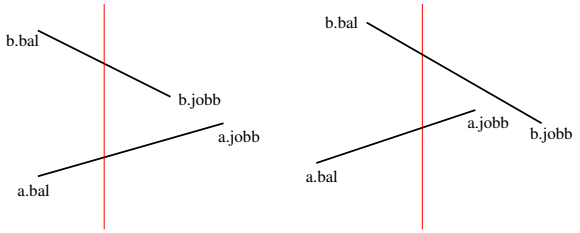
Ha `P` és `Q` tetszőleges poligonok, nem feltétlenül konvex, vagy monoton, akkor nem tudjuk lineáris időben meghatározni a metszetüket.
A megoldás visszavezethető arra a következőkben tárgyalt alapfeladat megoldására.

3. Metsző szakaszpár keresése

Bemenet: Adott egyenes szakaszoknak egy $S = (s_1, s_2, \ldots, s_n)$ halmaza.
Adjunk meg egy olyan s_a és s_b szakaszpárt, amelyeknek van közös pontja.

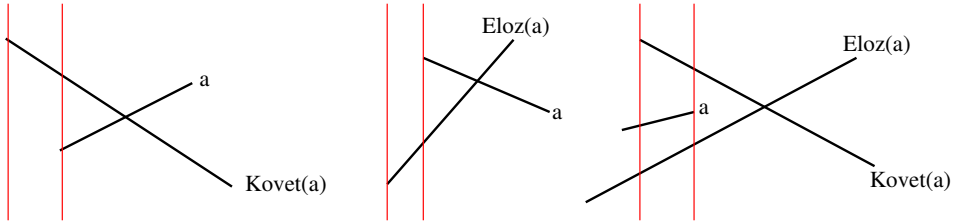
Megoldás

Alkalmazzuk a pásztázó egyenes módszerét: a pásztázó egyenesek az y-tengellyel párhuzamos egyenesek legyenek.
Minden `e` megállási helyhez tartozó állapothalmaz legyen azon szakaszok halmaza, amelyek metszik az `e` egyenest.
Feltesszük, hogy a szakaszok megadása olyan, hogy `S[i].bal.x < S[i].jobb.x` vagy `S[i].bal.x == S[i].jobb.x` és `S[i].bal.y < S[i].jobb.y`. Ha a bemenetben nem ez a helyzet, akkor cseréljük meg a szakasz két végpontját.
A pásztázó egyenes haladása során x-szerint növekvően, ha `S[i].bal.x = x`, akkor az `S[i]` szakasz bekerül az állapothalmazba, ha `S[i].jobb.x = x` akkor kikerül az állapothalmazból.
Az állapothalmaz lineárisan rendezett; `a < b` akkor és csak akkor teljesüljön, ha bármely olyan `e` pásztázó egyenes esetén, amely mindkettőt metszi, `a` metszéspontjának `y` koordinátája kisebb, mint `b` metszéspontjának `y` koordinátája.
A rendezési reláció megvalósításához nem kell kiszámítani a metszéspontot! Az alábbi ábra mutatja, hogy a szakaszok helyzete alapján hogyan lehet kiszámítani a reláció értékét.



6. ábra. Az `a < b` két esete

Az első esetben az $\overrightarrow{\texttt{a.bal, a.jobb}}$-tól `b.bal` és `b.jobb` balra van. A második esetben $\overrightarrow{\texttt{b.bal, b.jobb}}$-tól `a.bal` és `a.jobb` jobbra van.
Van egy extremális eset, amikor mindkét szakasz mindkét végpontja a pásztázó egyenesre esik. Ekkor az van előbb, amelyik bal végpontjának y koordinátája kisebb.
Jelölje `kovet(a)` `a` szakasz követőjét, `eloz(a)` pedig `a` közvetlen megelőzőjét a az `Allapot` rendezett halmazban. Az első metsző szakaszpár az alábbi három esetben következhet be.
Tehát ha egy szakaszt beteszünk az `Allapot` halmazba, akkor ellenőrizzük, hogy előzője vagy követője metszi-e (ha léteznek). Ha kiveszünk egy szakaszt az `Allapot` halmazból, akkor pedig azt ellenőrizzük, hogy az előzője és a követője metszik-e egymást. Ha metszést találunk, akkor befejezzük a keresést.
Megjegyzendő, hogy változtatás nélkül ez a módszer nem alkalmas arra, hogy az összes metsző párt meghatározzuk. Ugyanis két metsző szakasz egymáshoz való viszonya megváltozik a következő pásztázó egyenesre.



7. ábra

```
bool SzakaszokMetsz(vector<Szakasz> S, int& a, int& b){

  auto SzakaszRend=[S] (int i, int j){
    if(S[i].bal.x==S[i].jobb.x && S[j].bal.x==S[j].jobb.x)
    return S[i].bal.y<S[j].bal.y;
    return  Fordul(S[i].bal,S[i].jobb, S[j].bal)>=0 &&
    Fordul(S[i].bal, S[i].jobb, S[j].jobb)>=0 ||
    Fordul(S[j].bal,S[j].jobb, S[i].bal) <=0 &&
    Fordul(S[j].bal, S[j].jobb, S[i].jobb)<=0 ;
  };
  set<int, decltype(SzakaszRend)> Allapot(SzakaszRend);

  vector<pair<int ,bool>> Megallas;
  for(int i=0;i<S.size();i++){
    Megallas.pb({i,1});  Megallas.pb({i,0});
  }
  sort(Megallas.begin(), Megallas.end(),
    [S](pair<int,bool> a, pair<int,bool> b){
    return  S[a.first].bal.x<S[b.first].bal.x ||
    S[a.first].bal.x==S[b.first].bal.x &&
    S[a.first].bal.y<S[b.first].bal.y ||
    S[a.first].bal.x==S[b.first].bal.x &&
    S[a.first].bal.y==S[b.first].bal.y && a.second;});
```

```
40    for(auto p:Megallas){
41    if(p.second){//szakasz balvégpont
42      auto r=Allapot.insert(p.first).first;
43      auto kovet=next(r);
44      if(kovet!=Allapot.end() && SzakaszParMetsz(S[*r], S[*kovet])){
45        cout<<*r<<" "<<*kovet<<endl;
46        break;
47      }
48      auto eloz=prev(r);
49      if(eloz!=Allapot.end() && SzakaszParMetsz(S[*r], S[*eloz])){
50        cout<<*r<<" "<<*eloz<<endl;
51        break;
52      }
53    }else{//szakasz jobb-végpont
54      auto r=Allapot.find(p.first);
55      auto eloz=prev(r);
56      auto kovet=next(r);
57      if(eloz!=Allapot.end() && kovet!=Allapot.end() &&
58        SzakaszParMetsz(S[*eloz], S[*kovet])){
59          cout<<*eloz<<" "<<*kovet<<endl;
60          break;
61      }
62      Allapot.erase(r);
63    }
64  }
65  }
```

A megállási helyek rendezésének futási ideje `O(n log(n))`. Minden halmaz művelet `O(log(n))` idejű, ezért az algoritmus teljes futási ideje `O(n log(n))`

4. Utak feladat, CEOI'2020

Faország kormánya egy új úthálózatot szeretne építeni. Faországban $2N$ város van. Az úthálózat félig kész terve már tartalmaz N útszakaszt, ezek mindegyike két várost köt össze egy egyenes vonallal. Nincs olyan két útszakasz, amelynek lenne közös pontja (a végpontjaikat is beleértve).
A feladatod, hogy meghatározz $N - 1$ további útszakaszt, amelyek kielégítik az alábbi feltételeket:

1. Mindegyik új útszakasznak két várost kell összekötnie egy egyenes vonallal.
2. Ha két útszakasznak (újnak vagy réginek) van közös pontja, akkor ennek a pontnak mindkét útszakasz egy végpontjának kell lennie.
3. Az úthálózat az összes várost összeköti: bármely két városra létezik olyan útszakaszokból álló útvonal, amely a két várost összeköti.

Bemenet

A standard bemenet első sora a már létező útszakaszok N számát tartalmazza. A következő N sor mindegyike négy egész számot tartalmaz: x_1, y_1, x_2, y_2, ahol (x_1, y_1) és (x_2, y_2) az útszakasz végpontjainak a koordinátái.

Kimenet

A standard kimenetre $N - 1$ sort kell kiírni, mindegyikbe négy egész számmal, x_1, y_1, x_2, y_2, ahol (x_1, y_1) és (x_2, y_2) azon városok koordinátái, amelyek az új útszakasz végpontjai. Több megoldás esetén a program bármelyiket kiírhatja.

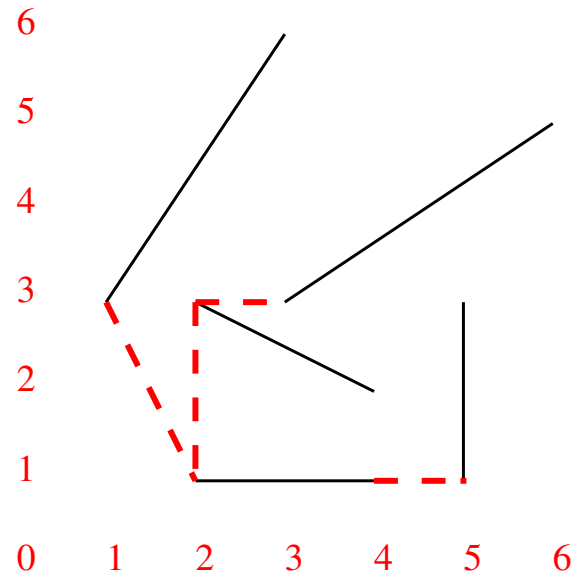
Példa

Bemenet

```
5
1 3 3 6
5 1 5 3
3 3 6 5
2 1 4 1
2 3 4 2
```

Kimenet

```
1 3 2 1
2 1 2 3
2 3 3 3
4 1 5 1
```



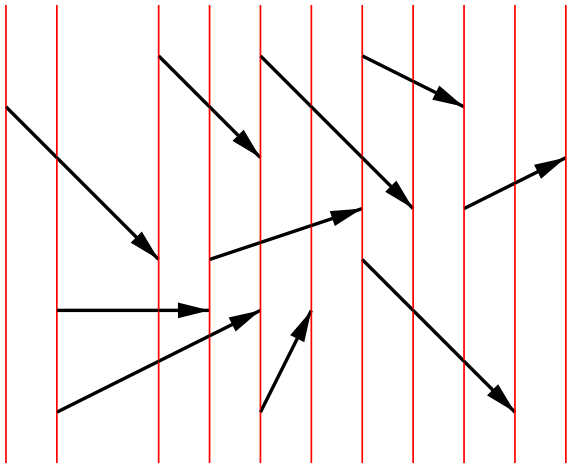
Korlátok

$2 \leq N \leq 10^5$
$-10^7 \leq x_i, y_i \leq 10^7$

Időlimit: 0.3 s

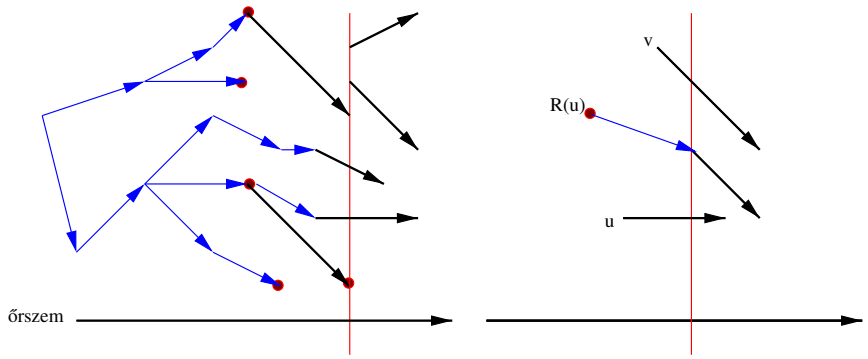
Memórialimit: 32 MiB

Megoldás



8. ábra

A szakaszokat tekintsük irányítottnak, az x koordinátájuk szerint.
Pásztázzunk egy olyan pásztázó egyenessel, amely párhuzamos az `y`-tengellyel, megállási pontjai az szakasz végpontok `x`-koordinátái.
Adott `x` megállási helyen az `Allapot` rendezett halmaz tartalmazza mindazon szakaszokat, amelyeknek van közös pontja a pásztázó egyenessel. A halmaz rendezési relációja legyen a metsző szakaszpárok megoldásánál használt rendezés.
A pásztázás során ha egy pont bal-végpontja egy szakasznak, akkor a szakasz bekerül `Allapot` halmazba, ha jobb-végpont, akkor kikerül.
Lépésről-lépésre építjük a megoldást fát, amikor egy szakasz bal-végpontját érinti a pásztázás, akkor azt a szakaszt egy új szakasszal bekötjük a fába.
Alkalmazzunk egy őrszem fiktív szakaszt, amely minden szakasz alatt van a szakaszok rendezése szerint.



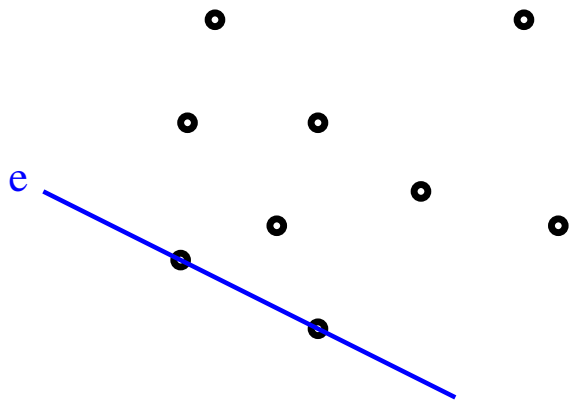
9. ábra. A piros pontok a csatalakozási pontok.

Ciklus invariáns

1. Minden olyan szakasz végpont, amely balra van a pásztázó egyenestől, benne van már az építendő fa megoldáskezdeményben.

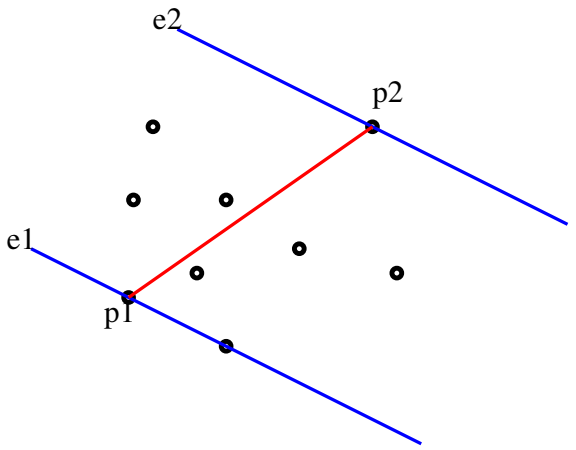
2. Legyen `u` és `v` két `Allapot` halmazban szomszédos szakasz, `u` van előbb. Minden szakaszhoz tartozik egy `R(u)` csatlakozási pont (valamely szakasz végpontja), amelyre teljesül, hogy bármely olyan `q` pont esetén, amely a pásztázó egyenesen van `u` és `v` között, az $\overline{\texttt{R(u), q}}$ szakasznak nincs közös pontja egyetlen szakasszal sem.

Kezdetben csak az őrszem van az Allapot halmazban, és az ehhez tartozó `R()` csatlakozó pont a legkisebb x-koordinátájú szakasz bal-végpont.
Tehát, ha egy pont bekerül `S`-be (azaz bal-végpont), akkor a beszúrás után megadhatjuk a megelőző `u` és követő `v` `S`-beli szakaszt, és a pontot a fában sz `R(u)` ponthoz kötjük, majd aktualizáljuk `R(u)` és az új szakaszhoz tartozó `R()` értékét.
Az algoritmus futási ideje `O(n log(n))`.



10. ábra

4.1. definíció. ***Támasztó egyenes***
Az e *egyenes támasztó egyenese a* P *ponthalmaznak, ha van olyan* $p \in P$ *amely rajta van az* e *egyenesen és nincs olyan* $p_1 \neq p_2 \in P$*, hogy* p_1 *és* p_2 *az* e *egyenes különböző oldalán lenne.*



11. ábra

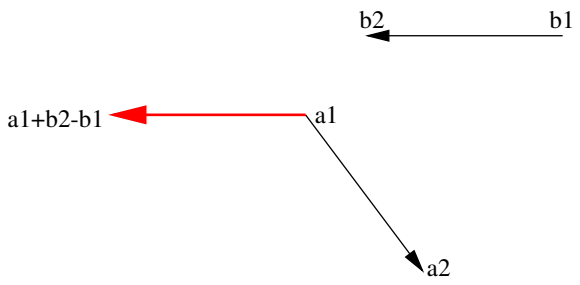
4.2. definíció. ***Átellenes támasztó egyenespár***
Az `e1` *és az* `e2` *egyenesek párhuzamosak, mindkettő támasztó egyenes és mindel pont a két egyenes között van.*

Definiáljuk a `Pont` adattípusra az összeadás és a kivonás műveletet.

```
Pont operator+(const Pont& p1, const Pont& p2) {
  return Pont(p1.x + p2.x, p1.y + p2.y, p1.azon);
}
Pont operator-(const Pont& p1, const Pont& p2) {
  return Pont(p1.x - p2.x, p1.y - p2.y, p1.azon);
}
```

Definiáljuk irányított szakaszok viszonyát úgy, hogy merre fordul $\overrightarrow{a1\ a2}$-től a $a1 + b2 - b1$ pont.Tehát

int Fordul(int a1, int a2, int b1, int b2) return Fordul(a1,a2, a1+b2-b1);



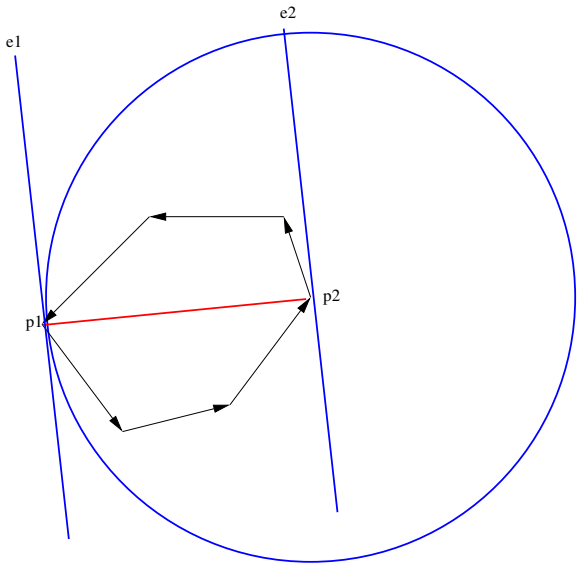
12. ábra

5. Zárt konvex poligon átmérője

Bemenet: $\mathtt{P} = (\mathtt{p}_0, \mathtt{p}_1, \ldots, \mathtt{p}_{n-1})$ zárt konvex poligon.
Számítsuk ki az átmérőjét. Pontosabban, adjunk meg olyan pontpárt, amelyek távolsága a lehető legnagyobb!

Megoldás

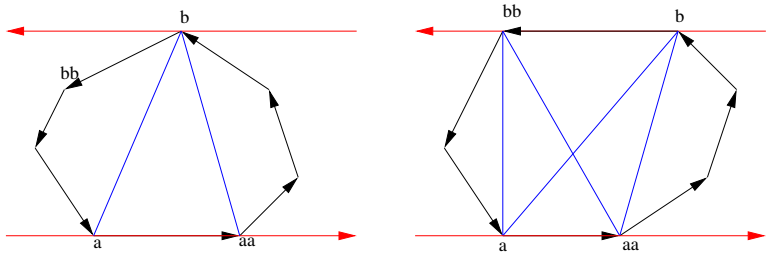
1. Észrevétel
Ha a `p1` és a `p2` csúcsok távolsága a legnagyobb, akkor van olyan `p1`-en átmenő és `e2` `p2`-n átmenő egyenes, amelyek átellenes támasztó egyenesek.



13. ábra

Vegyük azt a kört, amelynek középpontja `p2` és sugara `p1` és `p2` távolsága. A poligon minden pontja a körön belül van, tehát a `p1`-en, illetve `p2`-n átmenő, $\overline{\mathtt{p1\ p2}}$ szakaszra merőleges egyenesek átellenes támasztó egyenespár lesz.

Tehát megkapjuk az átmérőt, ha minden olyan $\overline{\mathtt{p1\ p2}}$ szakaszra, amelyek átellenes támasztó egyenespárokhoz tartoznak, kiszámítjuk a két pont távolságát, és vesszük a legnagyobbat. Természetesen elég a távolság négyzetét számolni, hogy elkerüljük a lebegőpontos aritmetika használatát.
Megmutatjuk, hogy legfeljebb `2 n` átellenes pontpár van, és ezeket elő tudjuk állítani `O(n)` időben. A poligon minden $\overrightarrow{\mathtt{a\ aa}}$ oldalához határozzuk meg azt a `b` csúcsot, amely pozitív forgásirány szerint az első olyan csúcs `aa`-tól, amelyhez van `b`-n átmenő egyenes, amely párhuzamos az $\overrightarrow{\mathtt{a\ aa}}$ oldallal. Két eset lehet:



14. ábra

- A $\overrightarrow{\mathtt{b\,bb}}$ szakasz nem párhuzamos $\overrightarrow{\mathtt{a\,aa}}$ oldallal.
  Ekkor `(a , b)` és az `(aa , b)` pontpárok átellenesek.
- A $\overrightarrow{\mathtt{b\,bb}}$ szakasz párhuzamos $\overrightarrow{\mathtt{a\,aa}}$ oldallal.
  Ekkor négy átellenes pontpár van: `(a , b)` és az `(aa , b)` `(a , bb)` és az `(aa , bb)`

Az első esetben `b` sz első olyan csúcs a forgásirány szerint, amelyre

```
Fordul(a, aa, b, bb) < 0
```

A második esetben

```
Fordul(a, aa, b, bb) = 0
```

5.0.1. Megvalósítás

```
#define kovet(i) (i+1)%n
#define eloz(i) (i-1+n)%n

using namespace std;
struct Pont{
  long long   x,y;
  int    azon;
  Pont(int a, int b, int c){x=a; y=b; azon=c;}
  Pont(){};
};
Pont operator+(const Pont& p1, const Pont& p2) {
  return Pont(p1.x + p2.x, p1.y + p2.y, p1.azon);
}
Pont operator-(const Pont& p1, const Pont& p2) {
  return Pont(p1.x - p2.x, p1.y - p2.y, p1.azon);
}
vector<Pont> P;

int Fordul(Pont A, Pont B, Pont C){
  long long Kereszt=(B.x-A.x)*(C.y-A.y)-(C.x-A.x)*(B.y-A.y);
  if (Kereszt <0)
  return -1;
  else if (Kereszt >0)
  return 1;
  else
  return  0;
}
int Fordul(int a1, int a2, int b1, int b2){
  return Fordul(P[a1],P[a2], P[a1]+P[b2]+P[b1]);
}
long long Tav(int i, int j){
  return (P[i].x-P[j].x)*(P[i].x-P[j].x)+
         (P[i].y-P[j].y)*(P[i].y-P[j].y);
}
```

```
void KonvexAtmero(vector<Pont>& P, int& p1, int& p2){
  int n=P.size();
  int a,aa,b,bb, ford;
  a=0; aa=1; b=1; bb=kovet(b);
  long long maxtav=0, tavab;
  while(b!=a){
    for(;;){
      ford=Fordul(a,aa, b,bb);
      if(ford<=0) break;
      b=bb; bb=kovet(b);
    }
    tavab=Tav(a,b);
    if(tab>maxtav){maxtav=tavab, p1=a; p2=b;}
    tab=Tav(aa,b);
    if(tab>maxtav){maxtav=tavab, p1=aa; p2=b;}

    if(ford==0){
      if(tab>maxtav){maxtav=tavab, p1=a; p2=b;}
      tab=Tav(aa,b);
      if(tab>maxtav){maxtav=tavab, p1=aa; p2=b;}
    }
    a=aa; aa=kovet(a);
  }
}
```

Az algoritmus minden $\overrightarrow{a\,aa}$ oldalra meghatározza a hozzá tartozó átellenes pontokat.
Az algoritmus futási ideje `O(n)`.
A módszer **forgó tolómérő (Rotating Caliper)** néven ismert a szakirodalomban. Az egyszerűbb, **görgetés** kifejezés is használatos erre a módszerre.

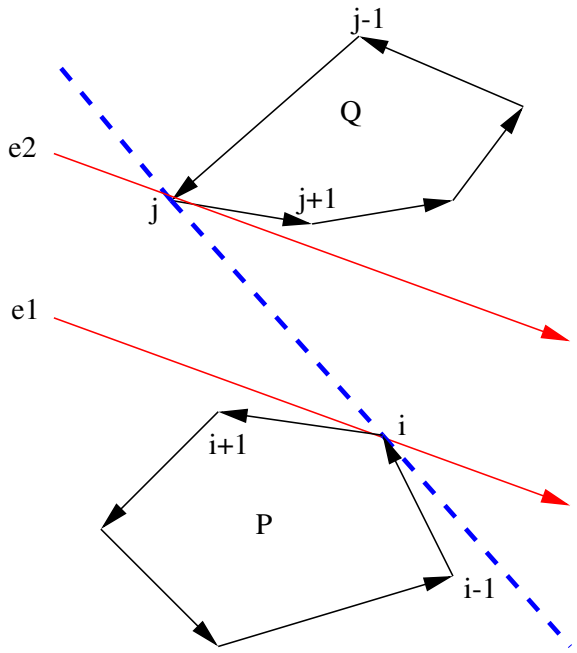
Következmény

`O(n log(n))` időben kiszámítható `n` pontot tartalmazó ponthalmaz legtávolabbi pontpárja.
A forgó tolómérő módszer alkalmazásával számos olyan probléma megoldható hatékonyan, amelyben konvex poligonok szerepelnek.

5.1. Két konvex poligon elválasztó egyenese

Bemenet: $P = p_0, p_1, \ldots, p_{n-1}$ és $Q = (q_0, q_1, \ldots, q_{m-1})$ konvex poligonok a csúcsai órajárással ellentétes felsorolásával. A két poligon nem metszi egymást.
Adjunk meg olyan $p_i \in P$ és $q_j \in Q$ pontokat, hogy az $e(p_i, q_j)$ egyenes elválasztja a két ponthalmazt. Tehát

nincs olyan két pontja a P poligonnak, amely az egyenes ellentétes oldalára esne, és nincs olyan két pontja a Q poligonnak, amely az egyenes ellentétes oldalára esne.
Észrevétel:
Ha P[i] és Q[j] megoldás pontok, akkor van olyan e1 P[i] -n átmenő és e2 Q[j] átmenő egyenes, amelyek



15. ábra

párhuzamosak és e1 támasztó egyenese P-nek, e2 támasztó egyenese Q-nak.
Tehát elegendő olyan P[i] és Q[j] pontpárok között keresni a megoldást, amelyekre teljesül a felni feltétel.
A görgetés módszerével O(n + m) időben elő tudjuk állítani az összes ilyen pontpárt.
Minden ilyen P[i] és Q[j] pontpárra egyszerűen eldönthető, hogy megoldás-e:

```
Fordul(P[i], Q[j], P[i − 1]) + Fordul(P[i], Q[j], P[i + 1]) > 0
```

és

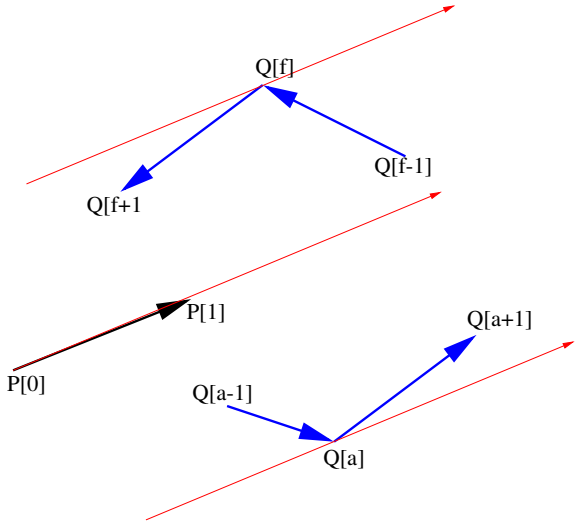
```
Fordul(P[i], Q[j], Q[j − 1]) + Fordul(P[i], Q[j], Q[j + 1]) < 0
```

P minden oldalához meghatározzuk azokat a Q-beli pontokat, amelyekhez van ellentétes támasztó egyenes, és alkalmazzuk a fenti feltételt.
A $\overrightarrow{P[0], P[1]}$ kezdő oldalhoz számítsuk ki azt a Q[a] és Q[f] pontokat, amelyekre illeszkedik $\overrightarrow{P[0], P[1]}$-vel párhuzamos támasztó egyenes. Az a-pont feltétele:

```
Fordul(P[0], P[1], Q[a − 1], Q[a]) + Fordul(P[0], P[1], Q[a], Q[a + 1]) > 0
```

Az f pontra vonatkozó feltétel:

```
Fordul(P[0], P[1], Q[f], Q[f − 1]) + Fordul(P[0], P[1], Q[f], Q[f + 1]) < 0
```



16. ábra

6. Drónfutár, IOI-válogató 2023

Manhattanben a legjobb pizzácó drónokat alkalmaz pizzafutárként. Regisztrálni lehet házhoz-szállításra, amely ingyenes, de cserébe biztosítani kell, hogy minden drón bármikor használhatja a helyet akkumulátorának feltöltésére, nem csak akkor, amikor oda szállít.
Ismerjük, hogy kik regisztráltak a házhoz szállítás szolgáltatásra, megadva a helyszín koordinátáit.

A drónok közlekedését szigorúan szabályozták, minden drón csak szögletesen közlekedhet, tehát mindig csak vagy függőlegesen, vagy vízszintesen haladhatnak. Tehát, ha az $A = (x_1, y_1)$ koordinátájú helyről a $B = (x_2, y_2)$ koordinátájú helyre megy, akkor $|x_1 - x_2| + |y_1 - y_2|$ hosszú utat tesz meg. Ha a drón akkumulátorának kapacitása nem elég ahhoz, hogy el tudjon menni egy helyről egy másikra, akkor közbülső helyek közbeiktatásával közlekedik, ahol mindig teljesen feltölti az akkumulátorát.
A drón teljesítménye az a legnagyobb `T` érték, ahány métert meg tud tenni egy feltöltéssel. Készíts programot, amely kiszámítja azt a legkisebb T teljesítményt, amely biztosítja, hogy a drón bármely helyről el tud menni bármely másik helyre, esetleg közbülső helyek igénybevételével!

Bemenet

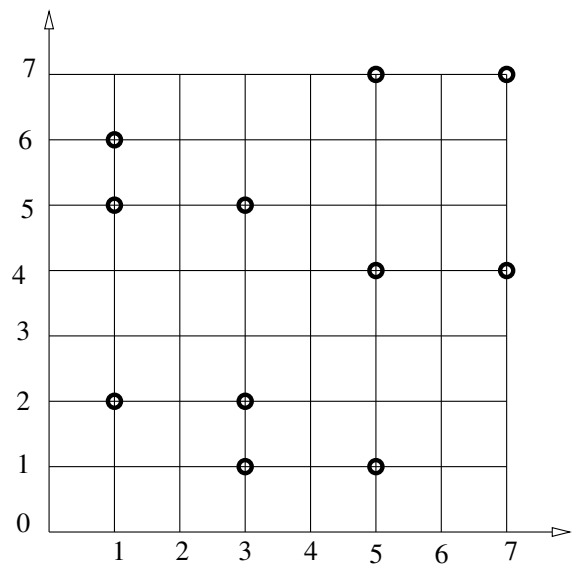
A `standard bemenet` első sorában a helyek `N` száma van.
A következő `N` sor mindegyike egy-egy hely x_i és y_i koordinátáit tartalmazza méterben mérve.

Kimenet

A `standard kimenetre` a drón szükséges legkisebb `T` teljesítményét kell írni!

Példa

Bemenet	Kimenet
11 1 2 1 5 7 4 3 2 3 5 5 1 5 4 3 1 5 7 7 7 1 6	3

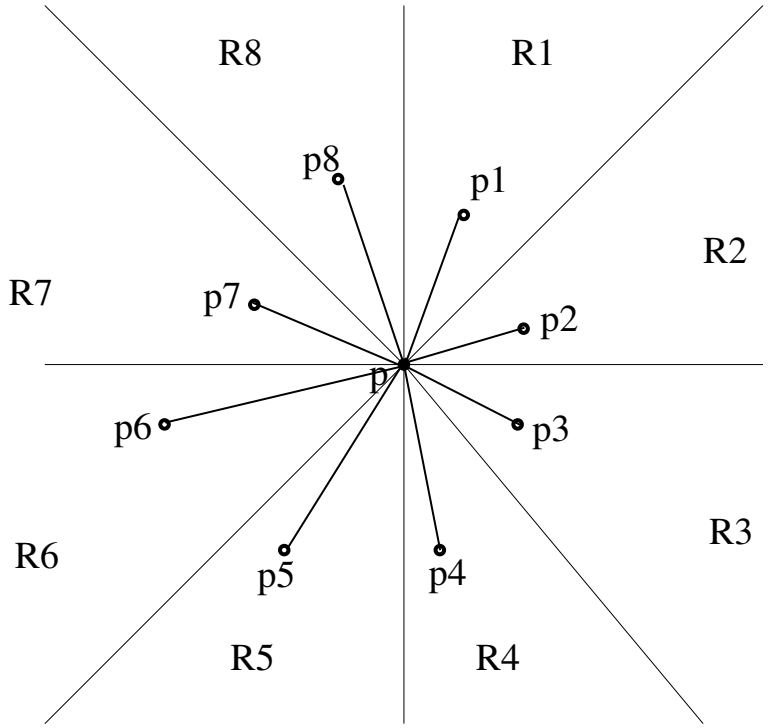


Korlátok

[$2 \le N \le 100\,000$] [$1 \le x_i, y_i \le 10^9$ minden $i = 1, 2, \ldots, N$-re] [$(x_i, y_i) \ne (x_j, y_j)$ minden $1 \le i \ne j \le N$-re]

Megoldás

Legyen G az az irányítatlan teljes gráf, amelynek pontjai a helyek és az élek súlya a pontpárok Manhattan-távolsága. Ekkor a megoldás egy minimális feszítőfa legnagyobb élének a súlya.
A standard megoldás lassú, $\mathcal{O}(n^2 \log n^2)$ a futási ideje.
Elegendő azonban a teljes gráf helyett olyan feszítőgráfot venni, amely minden p pont esetén a pont által meghatározott sík-nyolcadban lévő, hozzá legközelebbi pontot veszi élként. (Feszítőgráfnak nevezik az olyan részgráfot, amely tartalmaz legalább egy minimális feszítőfát.)



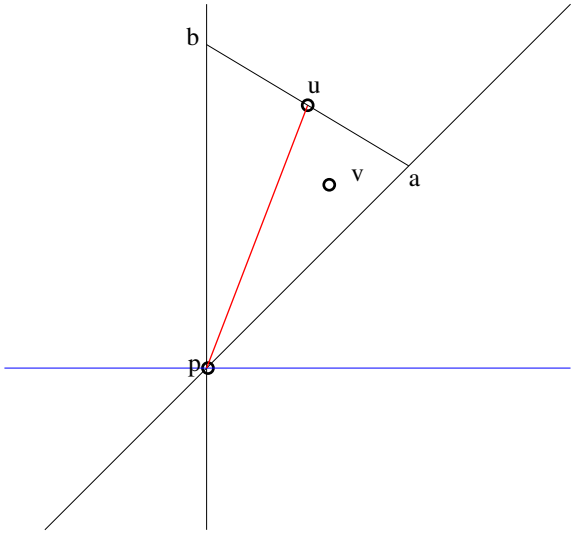
17. ábra

Minden p ponthoz legyen pi az R(i) sík-nyolcadban a p-hez legközelebbi (ha van). Legyen G8 az részgráfja G-nek, amely pontosan az (p, pi) éleket tartalmazza.

6.0.1. Állítás

A G8 minimális feszítőfája minimális feszítőfája lesz G-nek is.
Bizonyítás
Tegyük fel, hogy p, u egy éle a G egy minimális feszítőfájának és u nem a legközelebbi a p ponthoz a sík-nyolcadában. Legyen v az a pont a nyolcadban, amely p-hez legközelebbi, azaz Tav(u, v) < Tav(p, u).
Mivel v a (p, a, b) háromszögben van, de nem esik az (a, b) oldalra, ezét Tav(u, v) < Tav(p, u) is igaz.
Vegyük ki a minimális feszítőfából a (p, u) élet. Ekkor a feszítőfa szétesik két fára. v benne van valamelyikben. Ha abban van, amelyik a p pontot tartalmazza, akkor kössük össze a két fát a u, v éllel, ha az u-t tartalmazóban van, akkor pedig a (p, v) éllel. Mindkét esetben a minimális feszítőfa összköltsége csökkenne, ami ellentmond azon feltevésünknek, hogy u a legközelebbi p-hez.

Mivel G8 éleinek száma legfeljebb 8 n, ezért, ha O(n log(n)) időben elő tudjuk állítani a G8 gráfot, akkor a teljes algoritmus futási ideje is O(n log(n)) lesz.



18. ábra

Az `R1` sík-nyolcad esetére mutatjuk meg a feszítőgráf éleinek az előállítását.
Ábrázoljuk a bemeneti gráfot úgy, hogy az `i`-edik pontja koordinátái: `X[i]` és `Y[i]`.

$$\texttt{R1(i)} = \{\texttt{j} : \texttt{X[i]} < \texttt{X[j]} \text{ és } \texttt{X[j]} - \texttt{X[i]} < \texttt{Y[i]} - \texttt{Y[j]}\}$$

$$\texttt{R5(j)} = \{\texttt{i} : \texttt{X[i]} < \texttt{X[j]} \text{ és } \texttt{X[j]} - \texttt{X[j]} < \texttt{Y[i]} - \texttt{Y[j]}\}$$

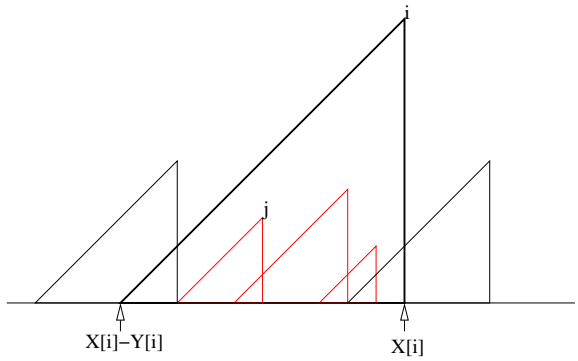
Tehát $\texttt{j} \in \texttt{R1(i)}$ akkor és csak akkor, ha $\texttt{i} \in \texttt{R5(j)}$
Hajtsunk végre pásztázást `x + y` érték szerint növekvően. Minden megállási pontban legyen `Aktiv` azon pontok halmaza, amelyeket már elértünk a pásztázás során és még nem határoztuk meg a ponthoz tartozó, `R1`-beli legközelebbi pontot.
Ekkor ha `i.` az aktuális megállási pont, és $\texttt{j} \in \texttt{R5(i)} \bigcap \texttt{Aktiv}$, akkor `i` a `j`-hez legközelebbi olyan pont, amely eleme `R1(j)`-nek. Tehát a `(j, i)` párt felvesszük élnek a feszítőgráfba és `j`-t töröljük `Aktiv`-ból, majd bevesszük `i`-t az `Aktiv`-ba.
Tehát az `Aktiv` halmazra mindig teljesül, hogy

$$(\forall \texttt{i, j} \in \texttt{Aktiv})\,(\texttt{X[i]} - \texttt{Y[i]} < \texttt{X[j]} - \texttt{Y[j]}) \text{ akkor } \texttt{X[i]} < \texttt{X[j]}$$

Ez azért teljesül, mert különben az `(i, j)` párt bevettük volna a feszítőgráfba és valamelyiket töröltük volna.
Ha rendezzük az `Aktiv` halmazt úgy, hogy $\texttt{i} < \texttt{j}$ akkor és csak akkor,
ha $\texttt{X[i]} - \texttt{Y[i]} < \texttt{X[j]} - \texttt{Y[j]}$, akkor hatékonyan megvalósíthatjuk a teendőt, amit tenni kell a megállás során.
Ugyanis, ha `i` a megállási pont, akkor

$$\texttt{R5(i)} \bigcap \texttt{Aktiv} = \{\texttt{j} : \texttt{X[i]} - \texttt{Y[i]} \leq \texttt{X[j]} - \texttt{Y[j]} \text{ és } \texttt{X[j]} \leq \texttt{X[i]}\}$$

Tehát az `Aktiv` halmazt ábrázolhatjuk `set < int >` típussal, vagy `map < int, int >` típussal, amelynek értékei: `Aktiv[X[i] - Y[i]] = i`.



19. ábra. `R5(i)` $\bigcap$ `Aktiv` elemei a pirosak

Ekkor a következő kóddal lehet elvégezni a teendőt ha `i.` az aktuális megállási pont:

```
auto it=Aktiv.lower_bound(X[i]-Y[i]);
while(it!=Aktiv.end() && X[it->second]<=X[i]){
  int j=it->second;
  Elek.emplace_back(abs(X[i]-X[j])+abs(Y[i]-Y[j]),i,j);
  it=Aktiv.erase(it);
}
Aktiv[X[i]-Y[i]]=i;
```

Így az elvégzendő teendő futási ideje az `i.` megállási pontban $\mathcal{O}$`(k log n)`, ha `k = |R5(i)` $\bigcap$ `Aktiv|`, tehát a teljes algoritmus futási ideje $\mathcal{O}$`(n log n)`.

Az látható, hogy ha elvégezzük a munkát az `R1` nyolcadra, akkor az `R5`-re már nem kell külön.
Az `(x, y)` $\longmapsto$ `(y, x)`, illetve a `(x, y)` $\longmapsto$ `(x, −y)` transzformációk alkalmazásával ugyanez a kód alkalmazható az `R2, R6, R3, R7` valamint az `R4, R8` nyolcadokkal is. Egy pásztázó menetben 2-2 nyolcad-párt tudunk feldolgozni. Így nagyon kompakt kódot adhatunk.

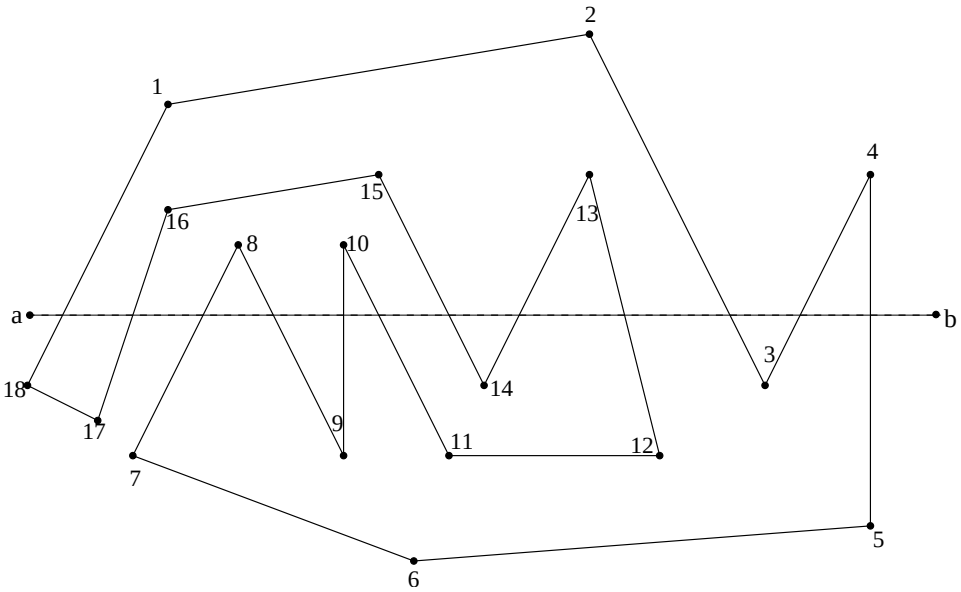
```
void FeszitoGraf(){
  vector<int> Sor(n);
  iota(Sor.begin(),Sor.end(),0);
  auto Rendrel=[&](int i, int j){return X[i]+Y[i]<X[j]+Y[j];};
  for(int u=0;u<2;++u){
    sort(Sor.begin(),Sor.end(),Rendrel);
    for(int v=0;v<2;++v){
      map<int,int> Aktiv;
      for(int i:Sor){
        auto it=Aktiv.lower_bound(X[i]-Y[i]);
        while(it!=Aktiv.end() && X[it->second]<=X[i]){
          int j=it->second;
          Elek.emplace_back(abs(X[i]-X[j])+abs(Y[i]-Y[j]),i,j);
          it=Aktiv.erase(it);
        }
```

```
            Aktiv[X[i]-Y[i]]=i;
        }
        swap(X,Y);
    }
    for(int i=0;i<n;++i)Y[i]=-Y[i];
  }
}
int Kruskal(){
  sort(Elek.begin(), Elek.end());
  int T=0, komp=n, hx,hy;
  UnioHolvan(n);
  for(auto [suly, x, y]:Elek){
    hx=Holvan(x); hy=Holvan(y);
    if(hx==hy) continue;
    T=suly;
    Unio(hx,hy);
    if(--komp==1) break;
  }
  return T;
}
```

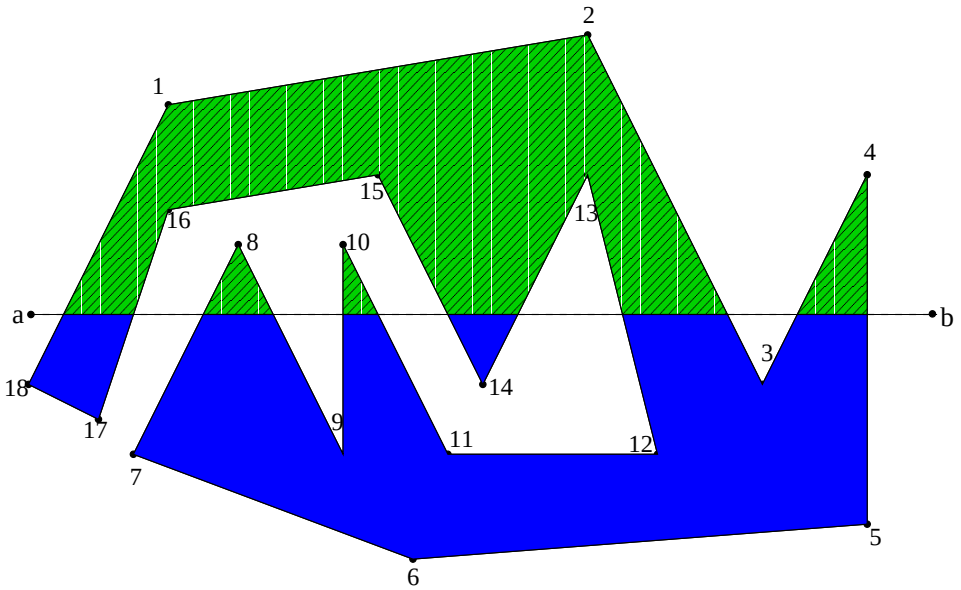
A feszítőgráf más módszerrel is előállítható $\mathcal{O}$ `(n log n)` időben, például osszdd meg és uralkodj stratégiával. Koordináta-kompresszió és szegmensfa használat is eredményes lehet.

7. Elektromos kerítés, CEOI'2005

Egy gazda villanypásztorral körülvett legelőn tartja a jószágait. Egy napon arról értesült, hogy a környéken utat fognak építeni. Kiment a legelőre és látta, hogy az építendő út nyomvonalát már kijelölték két oszloppal (`a` és `b`), és megállapította, hogy az út keresztülmegy a legelőjén.
Meg akarja állapítani, hogy az út hány összefüggő részre fogja bontani a legelőjét.
Pontosabban, azt akarja tudni, hogy hány rész lesz az út egyik, illetve másik oldalán.
Azonban nincs nála semmilyen mérőeszköz.



20. ábra



21. ábra. A bal oldalon 4, a jobb oldalon 3 ósszefüggő rész keletkezik.

Milyen információt tud szerezni a számításához?
Csak azt tudja megállapítani, hogy adott három oszlop milyen viszonyban van.

Pontosabban, a következőt tudja csinálni:
Elmegy az `i` oszlophoz, a `j` oszlop irányába nézve meg tudja állapítani, hogy egy harmadik oszlop, a `k` milyen irányban van; balra, jobbra, avagy egy egyenesbe esnek.

$$\texttt{Drift(i, j, k)} = \begin{cases} -1 & \text{ha i-ből j-felé nézve jobbra fordul} \\ 0 & \text{ha egy egyenese esnek} \\ +1 & \text{ha hez i-ből j-felé nézve balra fordul} \end{cases}$$

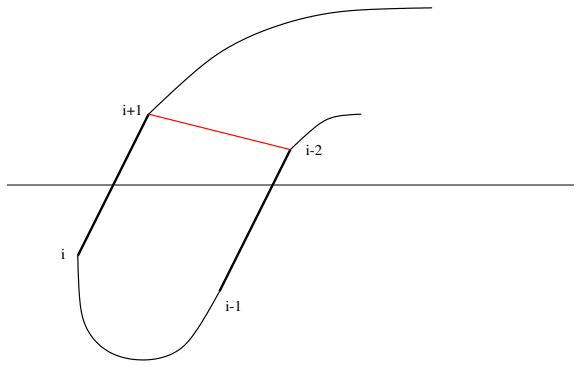
Ez a művelet könyvtári műveletként használható a megoldás elkészítésekor.

1. Megállapítás

Csak a `Drift` műveletet használva ki tudjuk számítani, hogy a kerítésnek hány oldala keresztezi az utat.

2. Megállapítás

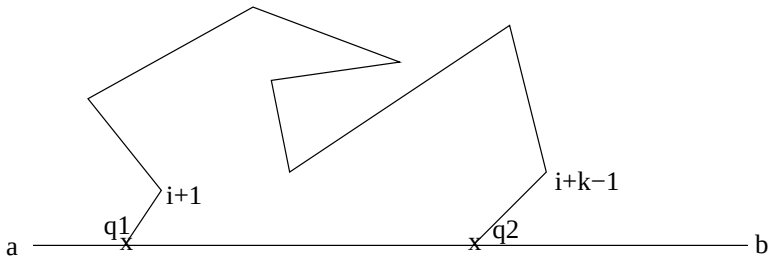
Ha a keresztező oldalak száma `m`, akkor összesen (az út két oldalán) `m/2 + 1` rész keletkezik.



22. ábra

Tehát elegendő a keresztező oldalak számát, és az út egyik oldalára eső részeket meghatározni.

Nevezzük *szegmensnek* a kerítés olyan összefüggő részét, amely két metszéspontot köt össze, de a kerítésrész többi oldalának egyike sem keresztezi az utat.



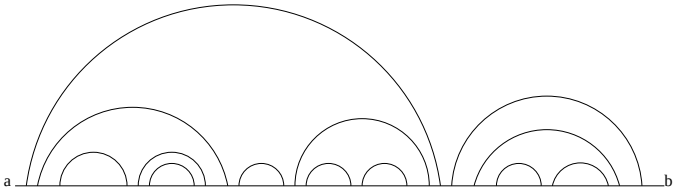
23. ábra

3. Megállapítás

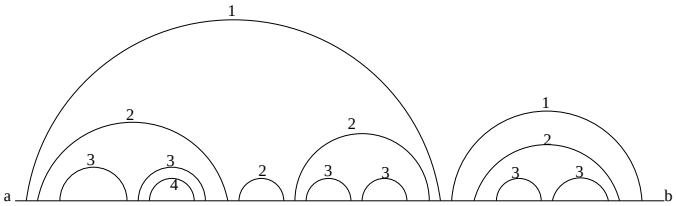
A szegmensek egymásba skatulyázottak.

Minden szegmensnek adjuk meg a szintszámát, kívülről befelé haladva.

4. Megállapítás

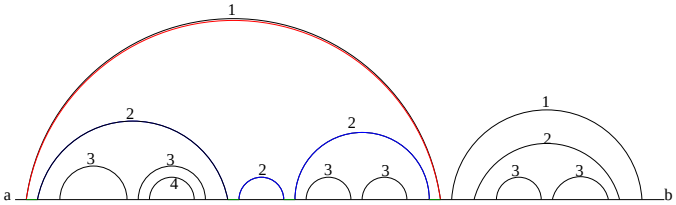


24. ábra



25. ábra

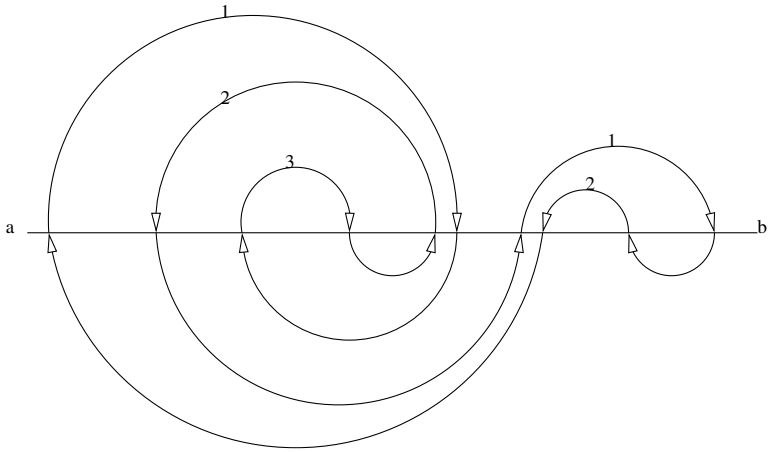
Az út adott oldalán annyi rész keletkezik, amennyi a páratlan szintszámú szegmensek száma.



26. ábra

Lássuk el a szegmenseket irányítással. Tegyük fel, hogy a szegmens q_1 végpontja közelebb van `a`-hoz, mint a q_2 végpontja. Ekkor az irányítása pozitív, ha az oszlopok sorszámozása növekvő, egyébként negatív.
Valójában a szegmensek azonosíthatók az `e(a, b)` egyenest metsző `i, i + 1` oldallal.

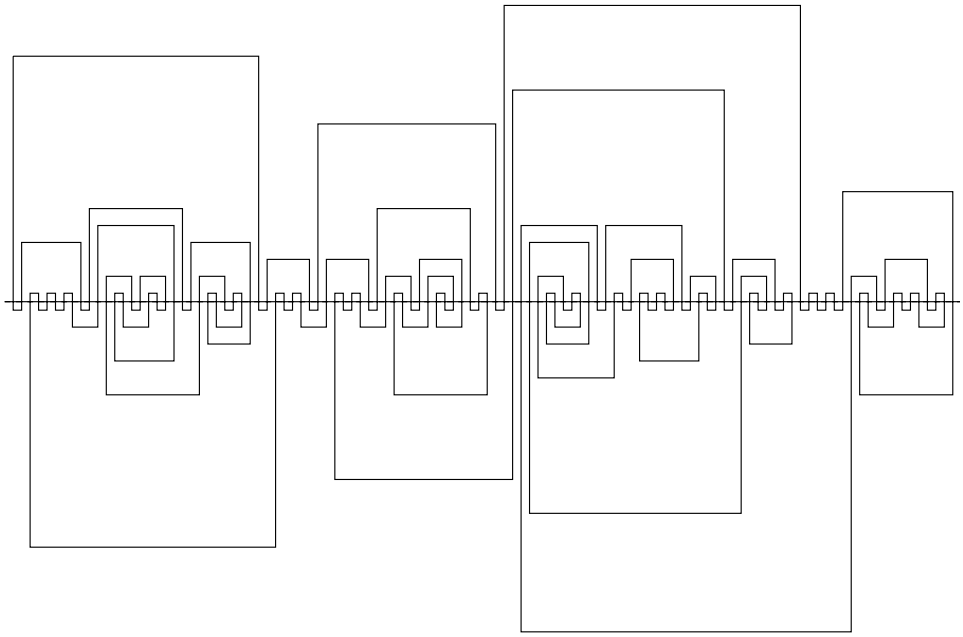
5. Megállapítás
A páratlan és páros szintszámú szegmensek irányítása ellentétes.



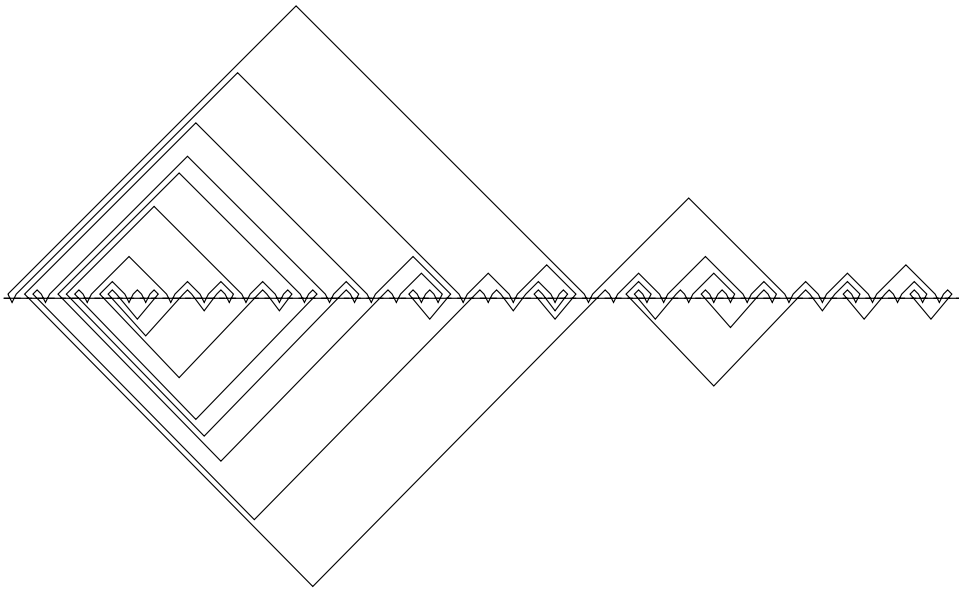
27. ábra

Első lépésként határozzuk meg azt az `i` oszlopot, amelyre teljesül, hogy az `i, i + 1` oldal metszi az `e(a, b)` egyenest és ez a metsző oldal van a legközelebb az `a` oszlophoz. Ez `O(n)` időben megtehető.
ettől az `i` oszloptól indulva, minden `i, i + 1` oldal esetén, ha az metszi az `e(a, b)` egyenest és `Drift(a, b, i) > 0`, akkor növeljük azt a `bal` számlálót, amely a bal oldali részek száma lesz.
Az algoritmus futási ideje a rendezés miatt `O(n)`.

Néhány minta bemenet, amelyek a versenyzők által használható segédprogrammal készültek.



28. ábra



29. ábra

8. 1. Feladat: Nyomtatott áramkör, CEOI'2012

https://mester.inf.elte.hu/

A Wikipedia szerint egy nyomtatott áramkört (NYÁK, angolul PCB) arra használnak, hogy a rajta kialakított elektromos áramkör alkatrészeit mechanikusan hordozza és közöttük villamos kötéseket biztosítson. A céged egy új elektronikus eszközt szeretne készíteni, ami NYÁK felhasználásával készül. A kívánt áramkör tervezése már részben kész van, és zárt poligon alakú. Az áramkör `1`-től `N`-ig számozott `N` darab pontból áll. Az `u` és `u + 1`-dik pont egy egyenes vezetékkel van összekötve, és az `N`-dik pont az elsővel (1) van összekötve egy egyenes vezetékkel. A vezetékek nem keresztezik egymást, azaz ha bármely két vezeték közös ponton osztozik, akkor annak a pontnak mindkét vezeték végpontjának kell lennie. Minden egyes pont helyét `x`- és `y`-koordinátákkal adjuk meg, az origó `(0, 0)` a NYÁK bal alsó sarka. Írj programot, ami meghatározza az áramkör összes olyan pontját, ami az origóból összeköthető egy olyan egyenes vezetékkel, aminek nincs közös pontja a poligon más pontjával magán a ponton kívül!

Bemenet

A `standard bemenet` első sorában áramkör pontjainak száma van $(3 \leq N \leq 100\,000)$ A következő `N` sor mindegyike egy-egy pont `x`- és `y`-koordinátáját tartalmazza $(0 < x, y \leq 1\,000\,000)$.

Kimenet

A `standard kimenetre` első sorába azon pontok számát kell írni, amelyek úgy köthetők össze egy egyenes vezetékkel az origóval, hogy ez a pont az egyetlen közös pont az áramkör vezetékeivel! A második sor ezeket a pontokat sorolja fel, növekvő sorrendben!

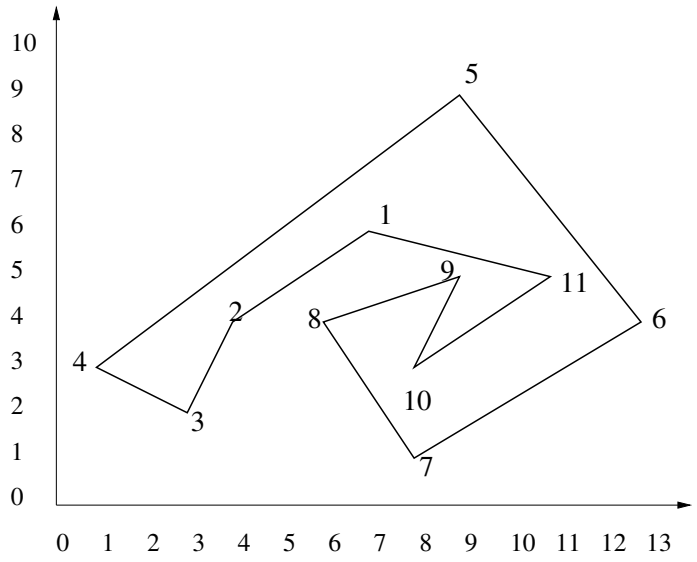
Példa

Bemenet

```
11
7 6
4 4
3 2
1 3
9 9
13 4
8 1
6 4
9 5
8 3
11 5
```

Kimenet

```
3
3 4 7
```



Korlátok

$3 \leq N \leq 100\,000$
$0 < x_i, y_i \leq 1\,000\,000$
Időlimit: 0.2 mp.
Memórialimit: 32MiB

Pontozás

A tesztesetek 10%-ban a pontok száma nem haladja meg az 1000-et.
Ha csak az első sor helyes, akkor a pontszámok 40%-a érhető el.

9. 2. Feladat: Elválasztás

https://mester.inf.elte.hu/ Geometriai algoritmusok, 50

Adott a síkon a `P` és a Q ponthalmaz.
Készíts programot, amely meghatároz egy-egy pontot a két ponthalmazban, amelyekre teljesül, hogy a két pont által meghatározott egyenes elválasztja a két ponthalmazt! Az elválasztás azt jelenti, hogy sem a `P`, sem a `Q` ponthalmazban nincs két olyan pont, amely az egyenes különböző oldalára esne.

Bemenet

A `standard bemenet` első sorában sorában két egész szám van, a `P` ponthalmaz pontjaninak `N1` $(2 \leq N1 \leq 100000)$ száma és a `Q` ponthalmaz pontjainak `N2` $(2 \leq N2 \leq 100000)$ száma. A további `N1` sor mindegyike a `P` ponthalmaz egy-egy pontjának `x`- és `y`-koordinátáját $(-1000000 \leq xi, yi \leq 1000000)$ tartalmazza. Az ezt követő `N2` sor mindegyike a `Q` ponthalmaz egy-egy pontjának `x`- és `y`-koordinátáját $(-1000000 \leq xi, yi \leq 1000000)$ tartalmazza.
Nincs olyan pont, amely mindkét ponthalmaznak eleme lenne.

Kimenet

A `standard kimenetre` első sorába azon első sorába azt az `i` és `j` sorszámot kell írni, amelyekre teljesül, hogy a `P` ponthalmaz `i`-edik pontja és a `Q` ponthalmaz `j`-edik pontja által meghatározott egyenes elválasztja a két ponthalmazt! Több megoldás esetén bármelyik megadható. Ha nincs megoldás, akkor a `0 0` számpárt kell kiírni!

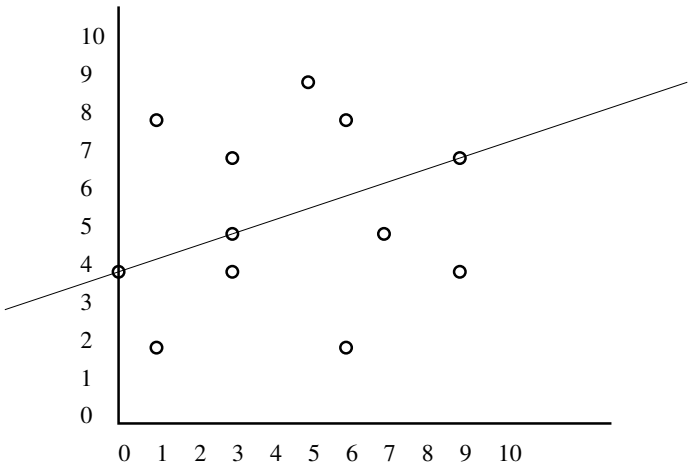
Példa

Bemenet

```
7 6
5 9
1 8
6 8
3 7
6 7
3 5
0 4
9 7
7 5
3 4
9 4
1 2
6 2
```

Kimenet

```
6 1
```



Korlátok

$2 \leq N1, N2 \leq 100\,000$
$-1\,000\,000 \leq x_i, y_i \leq 1\,000\,000$
Időlimit: 0.1 mp.
Memórialimit: 32MiB

Pontozás

A tesztesetek 20%-ban a pontok száma nem haladja meg az 1000-et.

10. Gyakorló feladatok

Utak: https://mester.inf.elte.hu/ CEOI2020
Úthálózatbővítés: https://mester.inf.elte.hu/ CEOI2012